Machine Learning Assisted Adaptive Chunk Caching for I/O Workloads

Review 3 final project report | Sashwath J | 24BAI0059

Abstract

I built a user-space chunk cache that calls a trained model from C when deciding whether to keep a fetched 4 KiB block. A cache miss still reads and returns the block; the model decides only whether it should occupy a cache slot afterward. Training used earlier portions of web-request streams, and testing used later portions or separate streams. With 512 slots, the model reduced direct-read calls by 1.0% to 7.5% against the same ARC or local S3-FIFO policy without ML. It also beat a second-touch admission rule in all eight matched comparisons. Checksums were identical for every policy on a given stream. Six of eight five-run median timing comparisons were faster with ML; two were slower. I also tested other cache sizes and a key-only stream, where the model was less reliable. The report gives the implementation, results, and conditions under which the read savings were observed.

Keywords: cache admission; machine learning; chunk caching; direct I/O; ARC; S3-FIFO; trace replay.

1 Introduction

When the cache is full, keeping one fetched block means evicting another. A wrong choice leads to another backing read later. I therefore focused on the admission decision made after a miss: return the requested block, then decide whether to keep it. The experiments replay the same request stream with the same number of 4 KiB slots under each policy. Direct-read calls are the main measure; request-loop time is measured separately.

Review 2 used CPU chunk scheduling, but its benchmark did not show a reliable improvement in total completion time. For Review 3 I moved the learned decision to I/O chunk caching, where the effect can be checked directly through backing-read counts. The earlier scheduling measurements are not used as cache results.

2 Problem statement objectives and contribution

Problem statement. For a fixed-capacity chunk cache, can an offline-trained prediction of near-term reuse reduce native backing-read calls when inserted before established replacement rules, while preserving every returned block?

The implementation follows six steps: prepare chronological request data; train and export the model; run its decision inside the C cache; compare each ML policy with its unchanged parent and a second-touch control; check returned data with checksums and independent replay; and measure reads and loop time. The contribution is the measured admission layer attached to two replacement policies. Learned caching itself is established work; the distinction here is the matched native experiment and the recorded cases where the rule fails.

3 Related work and research gap

Recent literature covers learned replacement, cache admission, simple eviction rules, and production deployment. CACHEUS adapts among replacement experts (Rodriguez et al., 2021). GL-Cache amortizes learning at a group level (Yang, Mao, et al., 2023). HALP augments a deployed CDN heuristic with preferences from a model (Song et al., 2023), while Baleen combines admission and prefetching under a flash-cache cost objective (Wong et al., 2024). S3-FIFO and SIEVE show that simple policies can be strong and inexpensive baselines (Yang, Zhang, et al., 2023; Zhang et al., 2024). 3L-Cache measures the CPU cost of learned caching at scale (Zhou et al., 2025). These studies motivate an explicit end-to-end cache metric, an unchanged parent comparison, and computation-cost reporting.

Table 1 lists 30 studies published or posted from 2021 to 2026 and identifies the part of this work each informed. Their datasets, cache sizes, object units, and metrics differ from mine, so I did not rank their reported percentages against my direct-read counts. Proceedings links, DOI records, and preprint status are given in the references.

No.	Study	Research focus	Link to project
1	Abdo et al. (2023)	A smart admission control and cache replacement approach in content delivery networks	Joint admission and replacement
2	Long (2022)	A Cache Admission Policy for Cloud Block Storage Using Deep Reinforcement Learning	Learned block admission
3	Fan et al. (2021)	PA-Cache: Evolving Learning-Based Popularity- Aware Content Caching in Edge Networks	Popularity-aware edge caching
4	Wang et al. (2023)	ACIC: Admission-Controlled Instruction Cache	Instruction-cache admission
5	Yang et al. (2022)	An End-to-End Automatic Cache Replacement Policy Using Deep Reinforcement Learning	Deep reinforcement learning replacement
6	Yan et al. (2021)	Learning from optimal caching for content delivery	Learning from offline decisions
7	Tian et al. (2022)	Efficient Federated DRL-Based Cooperative Caching for Mobile Edge Networks	Collaborative edge caching
8	Chen et al. (2023)	Darwin: Flexible Learning-based CDN Caching	CDN policy flexibility
9	Lyons & Rangaswami (2024)	To Cache or Not to Cache	Admission decision analysis
10	Yang, Pollen, et al. (2023)	CacheSack: Theory and Experience of Google's Admission Optimization for Datacenter Flash Caches	Datacenter flash admission
11	Torabi et al. (2024)	A Learning-Based Caching Mechanism for Edge Content Delivery	Edge content learning
12	Qiu et al. (2023)	FrozenHot Cache: Rethinking Cache Management for Modern Hardware	Hardware-aware management
13	Skachkov et al. (2024)	Learning-Augmented Online Caching: New Upper Bounds	Online learning theory
14	Liu et al. (2024)	SLAP: Segmented Reuse-Time-Label Based Admission Policy for Content Delivery Network Caching	Reuse-time admission
15	Dassanayake et al. (2024)	Multi-Agent Deep-Q Network-Based Cache Replacement Policy for Content Delivery Networks	Multi-agent replacement
16	Tran et al. (2024)	Federated Learning-Based Service Caching in Multi-Access Edge Computing System	Service-cache learning
17	Song et al. (2022)	RCM: A Remote Cache Management Framework for Spark	Remote cache management

No.	Study	Research focus	Link to project
18	Shahtouri et al. (2021)	Frozen Cache: Mitigating Filter Effect and Redundancy for Network of Caches	Network-cache filtering
19	Xia et al. (2026)	Learning-Augmented Heuristics: Simple, yet Smart, Robust and Interpretable Cache Eviction	Robust learned eviction
20	Zong et al. (2021)	Cocktail Edge Caching: Ride Dynamic Trends of Content Popularity with Ensemble Learning	Ensemble popularity learning
21	Neglia et al. (2021)	Similarity Caching: Theory and Algorithms	Similarity-based caching
22	Weerasinghe et al. (2022)	Reinforcement Learning Based Approaches to Adaptive Context Caching in Distributed Context Management Systems	Adaptive context caching
23	Rodriguez et al. (2021)	Learning Cache Replacement with CACHEUS	Adaptive replacement experts
24	Yang, Zhang, et al. (2023)	FIFO Queues Are All You Need for Cache Eviction	Simple queue-based replacement
25	Yang, Mao, et al. (2023)	GL-Cache: Group-level learning for efficient and high-performance caching	Learning overhead
26	Song et al. (2023)	HALP: Heuristic Aided Learned Preference Eviction Policy for YouTube Content Delivery Network	Production learned heuristic
27	Wong et al. (2024)	Baleen: ML Admission and Prefetching for Flash Caches	Admission with system cost
28	Zhang et al. (2024)	SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches	Simple strong baseline
29	Zhou et al. (2025)	3L-Cache: Low Overhead and Precise Learning-based Eviction Policy for Caches	Model CPU overhead
30	Lei & Shrivastav (2024)	Seer: Enabling Future-Aware Online Caching in Networked Systems	Future-aware online caching

Table 1. Thirty related studies from 2021–2026. The source title in each row is the traceable work; the last column states its relevance, not a comparative performance claim.

Several papers already use ML for admission or combine learning with replacement. My narrower question is whether one frozen admission model helps an existing cache when the requests and cache budget are held fixed. The second-touch control checks whether a simple admission filter would do the same job. I also changed capacity and removed features to see where the model stopped helping.

4 Methodology and system architecture

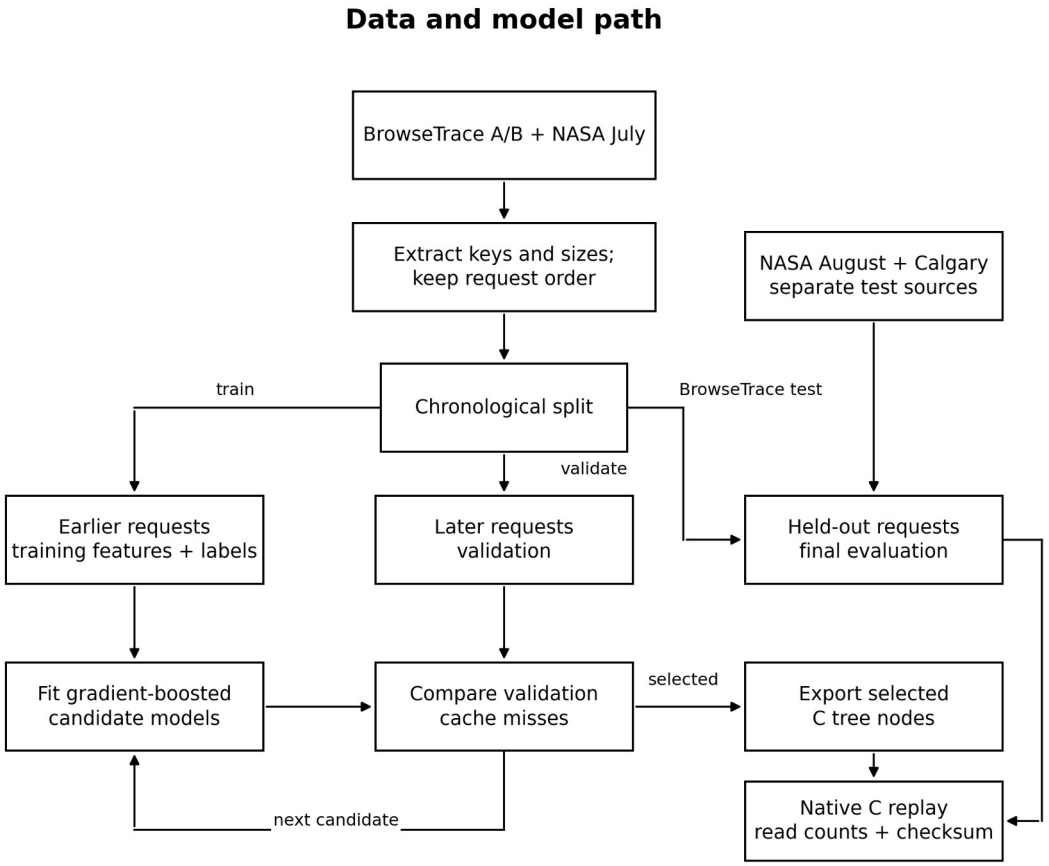


Figure 1. Chronological training, selection and native evaluation pipeline.

Earlier requests supply features and future-reuse labels for training. Later validation slices select a model and admission cutoff. Frozen C tree nodes enter the native replay; held-out BrowseTrace portions and separate NASA August and Calgary streams are used for final counting. The feedback arrow represents offline candidate selection, not online retraining.

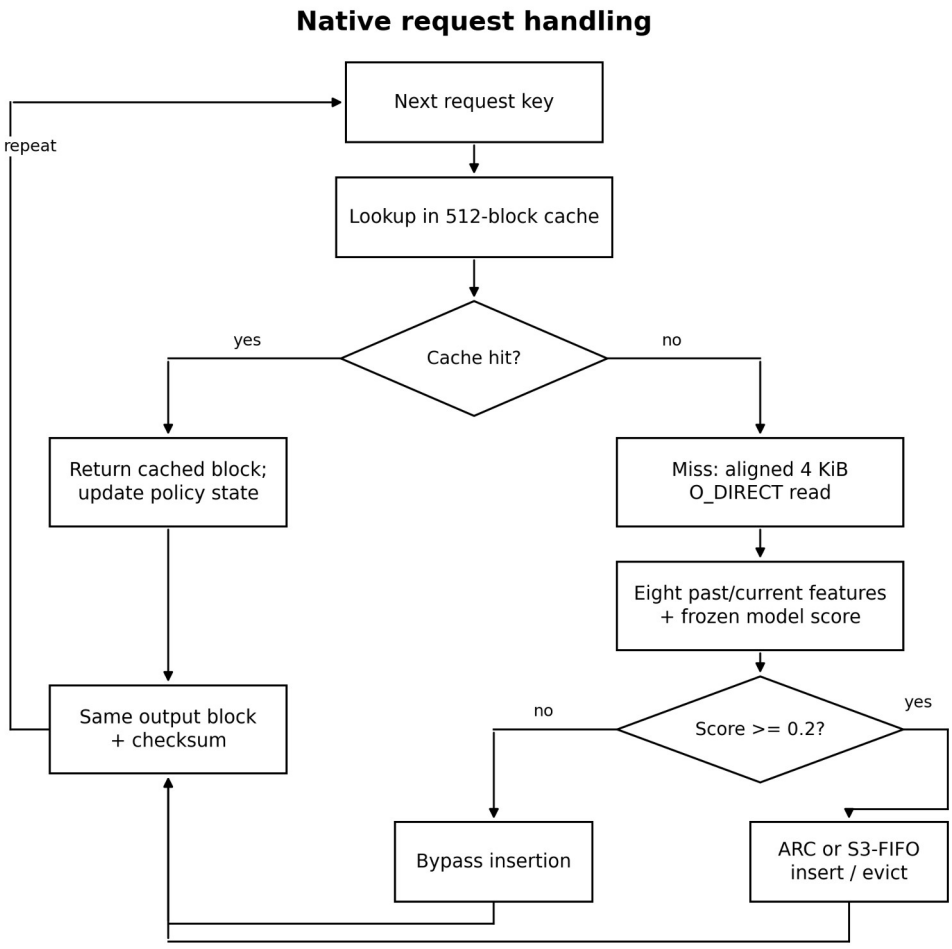


Figure 2. Native cache request path for the principal 512-entry, 0.2-cutoff run.

A hit returns the resident block. A miss always reads one aligned backing block, then evaluates the frozen model. A low score bypasses insertion; a high score passes the block to ARC or local S3-FIFO. Both branches return identical data and continue to the next request. The later capacity experiment changes the configured slot count and cutoff without retraining.

5 Data collection and preparation

The requests come from BrowseTrace A and B and historical NASA and Calgary web-server logs. NASA refers to a website log, not spacecraft data. I kept successful requests in their recorded order and converted each into a stable key with the metadata available at that request. The logs do not contain the original response bodies. For native tests, each key therefore maps to a deterministic 4 KiB backing block. This lets the program issue real aligned direct reads and check returned data, although the bytes are substitutes for the historical responses.

Source	Training	Validation	Final test
BrowseTrace A	49,473	16,491	16,491
BrowseTrace B	214,669	71,556	71,557
NASA July web	150,000	50,000	—
NASA August web	—	—	100,000
Calgary web	—	—	100,000

Table 2. Chronological data allocation; training has 414,142 requests and validation has 138,047. The unused NASA July tail is excluded from final claims.

The supervised target is one when the same key occurs within the next 1,024 requests of its trace slice and zero otherwise. The future window creates an offline training label; it is never read by the live cache. Each candidate model sees only the current request and earlier observations in its input features. The final streams are disjoint portions or sources. The later ML+S3-FIFO pairing on Calgary was selected after a plain-policy result was seen, so that pairing is an evaluation on held-out request data but not a fully blind policy-choice test.

6 Model and online feature construction

The fitted estimator is scikit-learn HistGradientBoostingClassifier with 100 boosting stages, at most 15 leaves per tree, at least 40 training examples per leaf, learning rate 0.07, L2 regularization 2, and seed 811. The exporter freezes 2,900 numeric tree nodes into the C evaluator. Model selection compared 40 versus 100 stages and admission thresholds 0.2, 0.4, and 0.6 using validation miss counts, retaining the 100-stage model with cutoff 0.2 for the principal 512-entry run. No fitting or Python prediction occurs during request serving.

Feature	Available information	Why used
log1p size	Current response metadata	Object magnitude
previous key count	Prior appearances of this key	Frequency
recency	Requests since previous same key	Locality
prior host count	Earlier host requests	Host activity
path length	Current URL path	Resource structure
static suffix flag	Current file extension	Static resource proxy
query flag	Current query string	Dynamic URL proxy
key length	Current full key	Key structure

Table 3. All eight model inputs are available when the request arrives or are maintained from earlier requests.

A score of at least 0.2 admits the fetched block in the main experiment. Below 0.2, the program returns the block but does not keep it. This threshold does not affect the hit lookup. The training label asks whether the key reappears within 1,024 requests; it does not predict the exact next-use position. Classification scores are therefore checked separately from actual cache reads.

7 Native modules and module workflows

Module	Input and operation	Verified output
Trace preparation	Ordered request keys and current metadata	Native input and counts
Training and selection	Earlier labels and validation replays	Frozen model and cutoff
C inference	Eight online numeric features	Admission score
Cache engine	Hit or aligned miss; ARC or local S3-FIFO	Returned block and read counter
Evaluation	Same streams and capacities across policies	Counts, checksums, timing JSON
Plot generation	Saved JSON and diagnostics	Reproducible PNG figures

Table 4. Major modules and inspectable outputs.

The native C process keeps at most 512 resident 4 KiB blocks in the main experiment, a 2 MiB data budget excluding cache metadata and model storage. It uses an aligned O_DIRECT read from a deterministic backing file on a miss, then scores, bypasses or inserts. ARC and local S3-FIFO are parent replacement choices; second-touch controls use the same parent but admit only after a key is seen again. Early tree traversal can stop once the remaining bounded contribution cannot change which side of the cutoff the score lies. Every policy produces the same checksum for a given trace, and independent Python replays reproduce the key native miss counts.

Figures 1–3 show where the modules meet. Figure 1 ends with a selected model exported for C. Figure 2 follows one request through lookup, read, scoring, and insertion or bypass. Figure 3 repeats the same test stream under each policy, checks the output, and saves the results. Plotting reads those saved results after the replays finish.

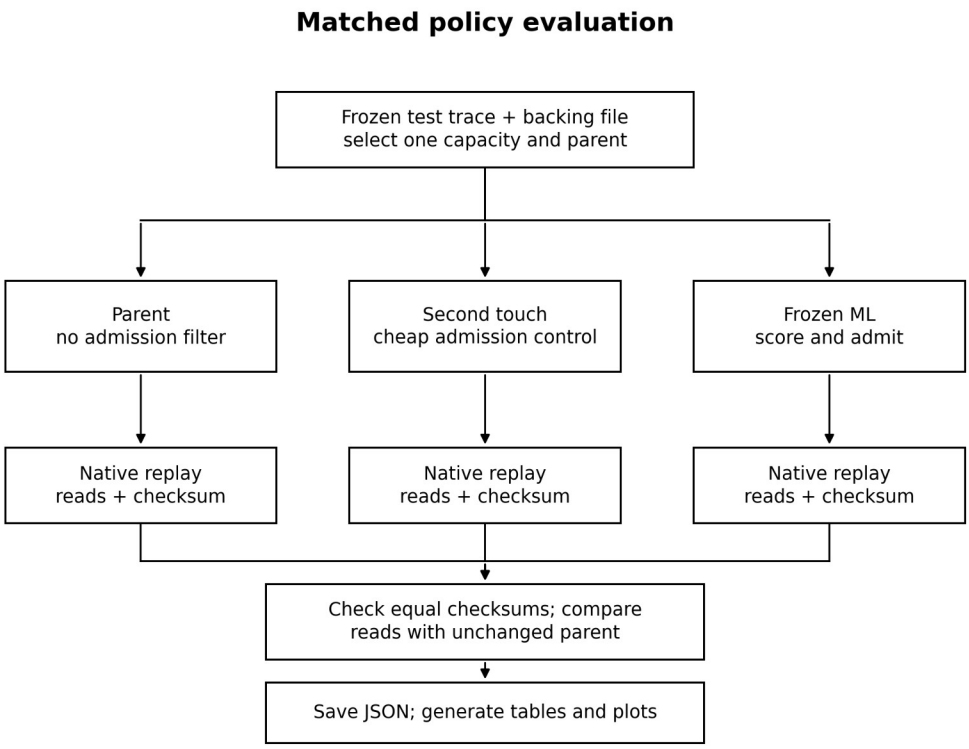


Figure 3. Matched native evaluation and result verification workflow.

Each test stream is replayed separately through an unchanged parent, its second-touch control and its ML admission variant at the same capacity. Direct-read calls and returned-block checksums are collected for every run. A failed checksum stops the comparison; matching outputs allow paired read-count and timing summaries to be saved. The result JSON is the input to the report figures.

8 Tools experimental setup and evaluation metrics

The implementation uses Python and scikit-learn for offline model fitting, C11 and GCC for native replay, Linux through WSL for direct I/O, and matplotlib for reproducible plots. The benchmark runs one policy at a time on the same ordered trace and capacity. The primary 512-entry counts are deterministic program-issued direct-read calls; the separate timing run stores five request-loop measurements per policy and reports their median. This isolates the cache behavior more directly than a single wall-clock run, while still exposing the CPU cost of model inference.

Metric	Calculation or meaning	Interpretation
Direct-read calls	Number of O_DIRECT calls on misses	Primary cache-efficiency measure
Read reduction	(parent − ML) / parent × 100	Matched policy effect
Hit rate	hits / total requests	Complement of miss fraction

Metric	Calculation or meaning	Interpretation
Median loop time	Median of five measured C loops	Observed runtime, not a guarantee
ROC AUC	Ranking of future-reuse labels	Classifier discrimination
Average precision	Precision–recall summary	Read with positive prevalence
Checksum equality	Same returned-block checksum per stream	Functional correctness check

Table 5. Monitored quantities and what each does and does not establish.

Each ML run uses the same requests, slot count, block size, and backing file as its parent. I compare ML+ARC with ARC and ML+S3-FIFO with local S3-FIFO. Crossing those pairs would change the replacement rule as well as admission, obscuring the model's effect. Published systems provide context, but I did not rerun their code under this setup.

9 Principal result at 512 entries

Stream	ARC	ML+ARC	2-touch ARC	S3-FIFO	ML+S3	2-touch S3
Browse A	7,430	7,104	7,828	7,773	7,336	8,322
Browse B	17,913	17,740	18,414	18,950	18,586	19,494
NASA Aug	8,792	8,130	9,094	8,349	8,048	8,551
Calgary	16,493	15,718	17,096	15,406	15,177	15,979

Table 6. Exact native direct-read calls, each on its own saved stream. Lower is better. The model beats both its unchanged parent and the second-touch admission control in all eight matched pairs at this capacity.

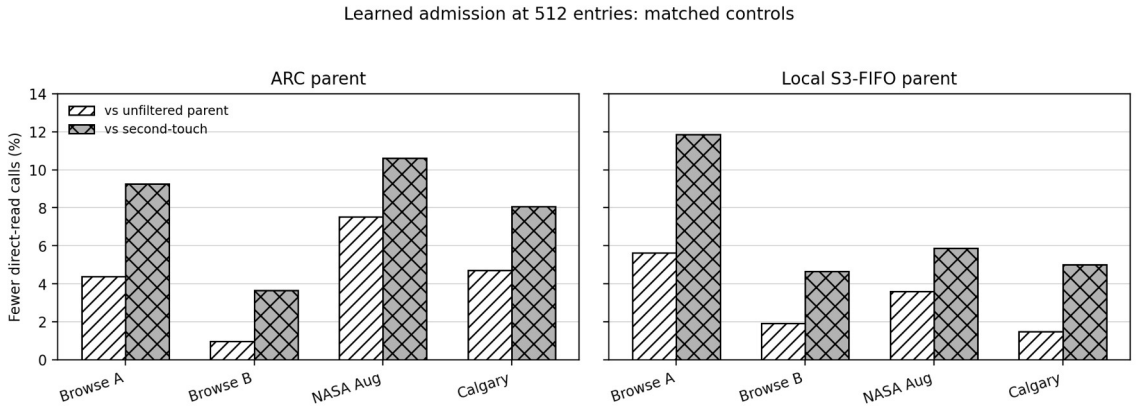


Figure 4. Read-call savings against the parent and second-touch control at 512 entries.

These bars are computed from the saved native admission-control experiment. Both ARC and local S3-FIFO panels use matched requests and capacity. ML+ARC saves 1.0% to 7.5% relative to ARC and also beats second-touch ARC on all four streams. The ML+S3-FIFO pairings likewise beat their corresponding parent and second-touch control. The simple admission control helps distinguish model benefit from the effect of filtering admissions generally.

I also ran LRU and 2Q on these streams. At 512 entries, LRU made 8,793, 19,713, 10,000 and 18,270 reads on Browse A, Browse B, NASA August and Calgary; 2Q made 7,655, 18,683, 9,670 and 17,807. Table 6 gives the more useful admission comparison because each ML policy sits beside its own parent. ML+ARC has the lowest count on Browse A and B; ML+S3-FIFO is lower on NASA August and Calgary.

10 Model check and elapsed time

Test stream	ROC AUC	Average precision	Reuse prevalence
Browse A	0.862	0.853	0.576
Browse B	0.816	0.920	0.722
NASA Aug	0.962	0.993	0.836
Calgary	0.889	0.965	0.761

Table 7. Frozen model on held-out streams. Higher average precision must be interpreted beside the high share of future-reuse labels.

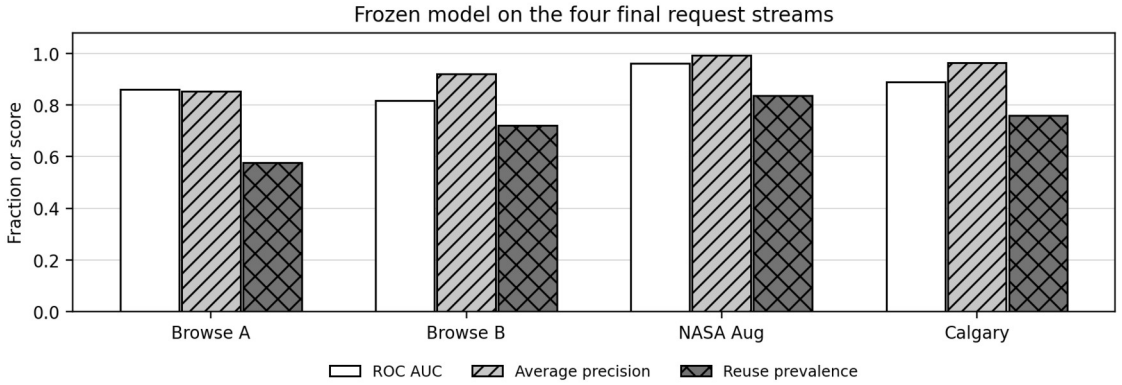


Figure 5. Frozen-model diagnostics for the four final streams.

The model was frozen before the later admission-control experiment. ROC AUC ranges from 0.816 to 0.962; average precision must be read beside reuse prevalence. Classifier quality does not substitute for the direct-read comparisons.

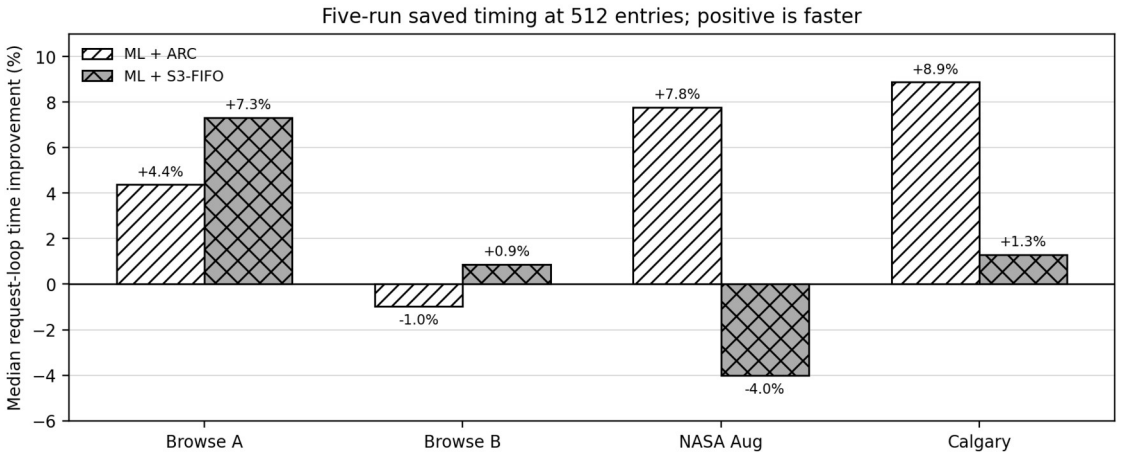


Figure 6. Five-run median request-loop timing from saved native repeats.

The later control and capacity sweeps used one run each, so this figure uses the separate five-run timing experiment. Six matched 512-entry comparisons have lower median time with ML; ML+ARC on BrowseTrace B is about 1.0% slower and ML+S3-FIFO on NASA August about 4.0% slower. The added read-count controls do not alter this timing measurement.

11 Capacity and cutoff sensitivity

To examine whether the main result persists beyond 2 MiB of data capacity, the same frozen model was rerun at 256, 512 and 1,024 entries, representing 1, 2 and 4 MiB. A fixed 0.2 cutoff beat the corresponding parent in 3 of 8 pairs at 256 entries, 8 of 8 at 512, and 4 of 8 at 1,024. The admission decision therefore depends on the cache budget. The additional selection script tested cutoffs 0.05, 0.1, 0.15, 0.2, 0.3, 0.4 and 0.5 on validation slices only, minimizing the worst ML-to-parent read ratio across those slices.

Cache entries	Data budget	ARC cutoff	S3-FIFO cutoff	Selected wins / ties / losses
256	1 MiB	0.05	0.30	4 / 3 / 1
512	2 MiB	0.30	0.20	8 / 0 / 0
1,024	4 MiB	0.10	0.10	6 / 2 / 0

Table 8. Validation-selected cutoff outcomes across the four test streams and two parents. These follow-up tests are exploratory because the capacity question arose after the principal results were examined.

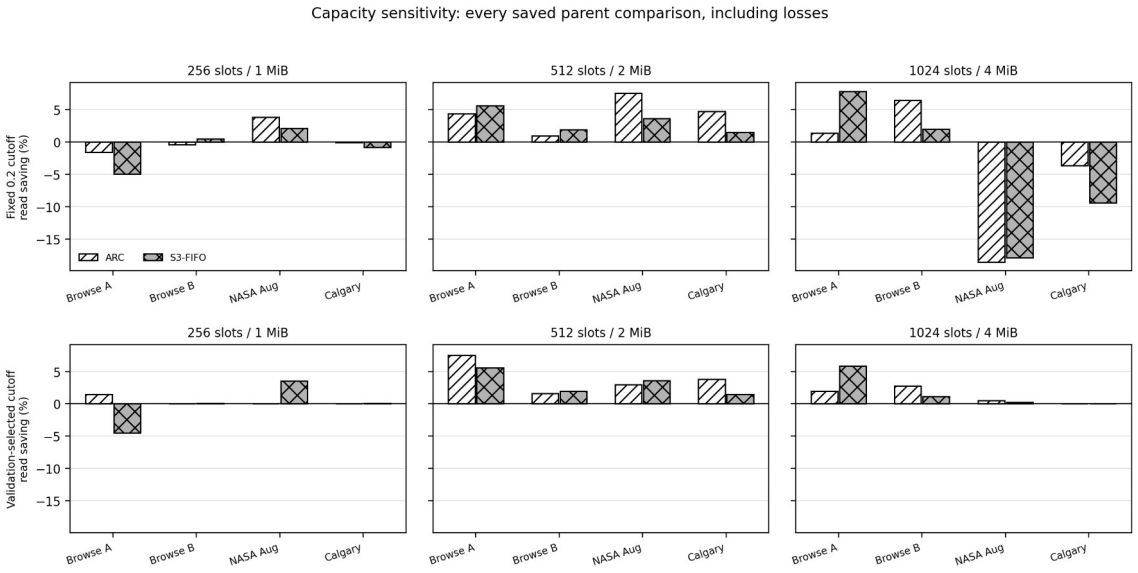


Figure 7. Capacity study using fixed and validation-selected admission cutoffs.

This chart is computed from the capacity-result JSON and includes every saved loss and tie. At 512 entries the fixed model is consistently helpful; at 256 and 1,024 entries it is mixed. Validation-selected thresholds produce 18 wins, 5 ties and 1 loss across 24 comparisons. The one loss is Browse A at 256 entries with ML+S3-FIFO, 4.53% more reads than its parent. These cells were run once, so they do not support a speed claim.

12 Comparison with published systems

Published system	Reported result in its own study	Why numbers are separate
HALP (2023)	9.1% mean peak byte-miss reduction; 1.8% CPU overhead	Production YouTube CDN; byte metric
Baleen (2024)	12% peak Disk-head Time reduction	Flash-cache backend cost metric
SIEVE (2024)	Up to 63.2% lower miss ratio than ARC	Different web trace collection
3L-Cache (2025)	6.4× LRU CPU cost at small cache size	Object/byte miss and CPU study
This prototype	1.0–7.5% fewer read calls at 512 slots	Four local streams; fixed 4 KiB blocks

Table 9. Published values are quoted only in their authors' experimental settings (Song et al., 2023; Wong et al., 2024; Zhang et al., 2024; Zhou et al., 2025). They are not head-to-head benchmarks and do not establish that this prototype outperforms the literature.

The papers also shaped the experiment. HALP and 3L-Cache report computation cost, so I measured loop time as well as reads. Baleen uses a clearly defined storage-cost target; here the target is direct-read calls. SIEVE and S3-FIFO motivated including strong simple cache policies. For each local replay, I kept the parent policy unchanged and compared it with second-touch and ML admission.

13 Robustness check and limitations

A separate sampled, anonymized key-only cache stream was used as a transfer stress test. It lacks the URL and object-size fields used in training. At 512 entries, ARC made 62,532 calls and ML+ARC made 69,720; local S3-FIFO made 61,628 and ML+S3-FIFO made 69,646. Thus the model is not suitable for this missing-feature workload without adaptation. This does not change the positive four-stream result; it bounds its generalization. The stress input is not a like-for-like original web-request trace, and its source bytes are not redistributed.

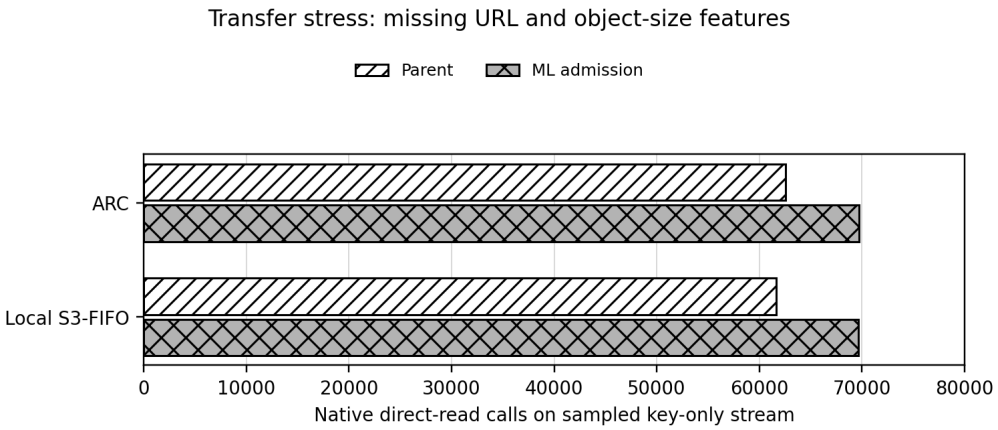


Figure 8. Key-only transfer stress test against both unchanged parents.

The chart uses a separate 100,000-key native replay. It shows more direct reads with ML when URL and size information are unavailable. This is a different input representation, so it is reported as a robustness boundary rather than combined with the four-stream result.

The replay uses substitute fixed-size blocks, one user-space process, and timing from one host. Local S3-FIFO is an implementation for this comparison, not the authors' optimized release. I counted read calls issued by the program, not physical disk transactions or network requests. The later cutoff study reused test streams I had already inspected, so I treat it as exploratory.

14 Completed implementation and reproducibility

The finished prototype contains trace conversion, offline training, frozen-model export, C inference and cache policies, native direct-I/O replay, checksum validation, result recording and figure generation. The saved 512-entry comparison, second-touch controls, capacity sweep, model diagnostics and key-only transfer check are inspectable. The main counts were independently checked against Python replay and can be reproduced by the native live demonstration. Timing may move with host load; read counts and same-stream checksums are deterministic for these fixed inputs.

The original Review 2 CPU scheduling study motivated the change in scope to I/O chunk caching. Review 3 reports the implemented cache system and its own results; the old scheduling results are not reused as cache evidence. The transition is documented in REVIEW2_TO_FINAL_TRANSITION.md in the project materials.

15 Conclusion and future scope

The C cache uses a frozen gradient-boosting model to decide which fetched blocks to retain. With 512 slots, it made fewer backing-read calls than its unchanged parent in all eight matched comparisons on four streams. The reductions ranged from 1.0% to 7.5%; six of eight five-run median loop times were lower. Second-touch admission did not match the ML result in these runs. At other capacities the fixed cutoff was less consistent, and on the key-only stream ML increased reads. The measured improvement applies to the tested workloads and 512-slot setting; it is not a general speed or transfer guarantee.

Future research can test a capacity-aware cutoff or a fallback that disables ML when required URL and size features are absent, then validate those choices on untouched streams. A production study would also need

original payloads, concurrent clients, kernel or storage-stack integration, and device-level I/O measurement. These are extensions beyond the completed user-space prototype, not assumptions used to interpret the present results.

16 Source code and supporting evidence

The project files are available in the public repository below. Table 10 identifies the code and saved evidence most useful for checking the result. The plots are generated from JSON records. Source manifests record where the traces came from; third-party source bytes are not included when they cannot be redistributed.

Project repository: https://github.com/Sashwath-sash/ml-adaptive-chunk-scheduling

Item	File or directory	What to inspect
Native engine	cache_native.c	C request loop, direct reads, inference and cache policies
Model fit/export	train_three_source_cache.py; export_cache_native.py	Training choices and frozen numeric trees
Demo and checks	demo_hybrid_final.py; verify_admission_extension.py	Saved evidence and fresh replay assertions
Inputs	cache_native_input_calgary_v1/; source manifests	Ordered requests, model input and provenance
Results	results_hybrid_native.json; results_admission_controls_2026-10-27.json	Policy counts and checksums
Sensitivity	results_admission_controls_256_2026-10-27.json; results_admission_controls_1024_2026-10-27.json	Capacity outcomes
Figures and audit	final_review_materials/build/; research_extension_2026-10-27/	Figure scripts and research status

Table 10. Source and evidence index for the Review 3 submission. The repository and the accompanying project folder supply the full implementation; this report identifies the files needed for inspection.

17 Final working demonstration

Open a terminal in the project root. Run python demo_hybrid_final.py to verify the saved four-stream experiment. Its first two PASS lines check inputs, results and checksums, followed by the four-row table in this report. Run python verify_admission_extension.py to check the controls, capacity-selection record and key-only stress counts; success prints a PASS line. These checks use saved evidence and do not measure new wall time.

For a fresh native demonstration on the prepared Windows/WSL machine, run python demo_hybrid_final.py --live calgary. The script compiles cache_native.c with GCC in Linux, replays the Calgary test trace with ARC, ML+ARC, local S3-FIFO and ML+S3-FIFO, and asserts equal returned-data checksums. Expected direct-read counts are 16,493, 15,718, 15,406 and 15,177 respectively, matching Table 6. The loop-time number printed beside each count is a new observation and need not match the saved five-run medians. The 4 KiB backing reads occur in the Linux native executable; Python coordinates the demonstration and verification.

References

Abdo, L., Ahmad, I., & Abed, S. (2023). A smart admission control and cache replacement approach in content delivery networks. Cluster Computing. https://doi.org/10.1007/s10586-023-04095-7

Chen, J. K., Sharma, N., Khan, T., Liu, S., Chang, B., Akella, A., Shakkottai, S., & Sitaraman, R. K. (2023). Darwin: Flexible Learning-based CDN Caching. ACM SIGCOMM Conference (SIGCOMM). https://doi.org/10.1145/3603269.3604863

Dassanayake, J. K., Wang, M., Hameed, M. Z., & Yang, N. (2024). Multi-Agent Deep-Q Network-Based Cache Replacement Policy for Content Delivery Networks. Future Internet. https://doi.org/10.3390/fi16080292

Fan, Q., Li, X., Li, J., He, Q., Wang, K., & Wen, J. (2021). PA-Cache: Evolving Learning-Based Popularity-Aware Content Caching in Edge Networks. IEEE Transactions on Network and Service Management. https://doi.org/10.1109/tnsm.2021.3053645

Lei, J., & Shrivastav, V. (2024). Seer: Enabling Future-Aware Online Caching in Networked Systems. 21st USENIX Symposium on Networked Systems Design and Implementation. https://www.usenix.org/conference/nsdi24/presentation/lei

Liu, K., Wu, K., Wang, H., Zhou, K., Wang, P., Zhang, J., & Li, C. (2024). SLAP: Segmented Reuse-Time-Label Based Admission Policy for Content Delivery Network Caching. ACM Transactions on Architecture and Code Optimization. https://doi.org/10.1145/3646550

Long, F. (2022). A Cache Admission Policy for Cloud Block Storage Using Deep Reinforcement Learning. Lecture notes in electrical engineering. https://doi.org/10.1007/978-981-19-3927-3_46

Lyons, S., & Rangaswami, R. (2024). To Cache or Not to Cache. Algorithms. https://doi.org/10.3390/a17070301

Neglia, G., Garetto, M., & Leonardi, E. (2021). Similarity Caching: Theory and Algorithms. IEEE/ACM Transactions on Networking. https://doi.org/10.1109/tnet.2021.3126368

Qiu, Z., Yang, J., Zhang, J., Li, C., Ma, X., Chen, Q., Yang, M., & Xu, Y. (2023). FrozenHot Cache: Rethinking Cache Management for Modern Hardware. https://doi.org/10.1145/3552326.3587446

Rodriguez, L. V., Yusuf, F., Lyons, S., Paz, E., Rangaswami, R., Liu, J., Zhao, M., & Narasimhan, G. (2021). Learning Cache Replacement with CACHEUS. 19th USENIX Conference on File and Storage Technologies. https://www.usenix.org/conference/fast21/presentation/rodriguez

Shahtouri, S. M., Rezazad, M., & Ma, R. T. B. (2021). Frozen Cache: Mitigating Filter Effect and Redundancy for Network of Caches. IEEE Access. https://doi.org/10.1109/access.2021.3111220

Skachkov, D., Ponomaryov, D. K., Dorn, Y., & Demin, A. V. (2024). Learning-Augmented Online Caching: New Upper Bounds. arXiv (Cornell University). https://doi.org/10.48550/arxiv.2410.01760

Song, Y., Yu, J., Li, B., Li, H., He, X., Wang, J., & Zhai, R. (2022). RCM: A Remote Cache Management Framework for Spark. Applied Sciences. https://doi.org/10.3390/app122211491

Song, Z., Chen, K., Sarda, N., Altınbüken, D., Brevdo, E., Coleman, J., Ju, X., Jurczyk, P., Schooler, R., & Gummadi, R. (2023). HALP: Heuristic Aided Learned Preference Eviction Policy for YouTube Content Delivery Network. 20th USENIX Symposium on Networked Systems Design and Implementation. https://www.usenix.org/conference/nsdi23/presentation/song-zhenyu

Tian, A., Feng, B., Zhou, H., Huang, Y., Sood, K., Yu, S., & Zhang, H. (2022). Efficient Federated DRL-Based Cooperative Caching for Mobile Edge Networks. IEEE Transactions on Network and Service Management. https://doi.org/10.1109/tnsm.2022.3198074

Torabi, H., Khazaei, H., & Litoiu, M. (2024). A Learning-Based Caching Mechanism for Edge Content Delivery. ACM/SPEC International Conference on Performance Engineering (ICPE). https://doi.org/10.1145/3629526.3645037

Tran, T. P., Tran, A. H. N., Nguyen, T. M., & Yoo, M. (2024). Federated Learning-Based Service Caching in Multi-Access Edge Computing System. Applied Sciences. https://doi.org/10.3390/app14010401

Wang, Y., Chang, C., Sivasubramaniam, A., & Soundararajan, N. K. (2023). ACIC: Admission-Controlled Instruction Cache. Proceedings - International Symposium on High-Performance Computer Architecture/Proceedings. https://doi.org/10.1109/hpca56546.2023.10071033

Weerasinghe, S., Zaslavsky, A., Loke, S. W., Abken, A., & Hassani, A. (2022). Reinforcement Learning Based Approaches to Adaptive Context Caching in Distributed Context Management Systems. arXiv (Cornell University). https://doi.org/10.48550/arxiv.2212.11709

Wong, D. L., Wu, H., Molder, C., Gunasekar, S., Lu, J., Khandkar, S., Sharma, A., Berger, D. S., Beckmann, N., & Ganger, G. R. (2024). Baleen: ML Admission and Prefetching for Flash Caches. 22nd USENIX Conference on File and Storage Technologies. https://www.usenix.org/conference/fast24/presentation/wong

Xia, H., Nixon, W., Marthen, B. D., Bhandari, P., & Yang, J. (2026). Learning-Augmented Heuristics: Simple, yet Smart, Robust and Interpretable Cache Eviction. arXiv (Cornell University). https://doi.org/10.48550/arxiv.2608.27975

Yan, G., Li, J., & Towsley, D. (2021). Learning from optimal caching for content delivery. ACM International Conference on emerging Networking EXperiments and Technologies (CoNEXT). https://doi.org/10.1145/3485983.3494855

Yang, Z., Wang, F., Shi, Z., & Feng, D. (2022). An End-to-End Automatic Cache Replacement Policy Using Deep Reinforcement Learning. Proceedings of the International Conference on Automated Planning and Scheduling. https://doi.org/10.1609/icaps.v32i1.19840

Yang, T., Pollen, S., Uysal, M., Merchant, A., Wolfmeister, H., & Khalid, J. (2023). CacheSack: Theory and Experience of Google's Admission Optimization for Datacenter Flash Caches. ACM Transactions on Storage. https://doi.org/10.1145/3582014

Yang, J., Zhang, Y., Qiu, Z., Yue, Y., & Rashmi, K. V. (2023). FIFO Queues Are All You Need for Cache Eviction. 29th ACM Symposium on Operating Systems Principles. https://doi.org/10.1145/3600006.3613147

Yang, J., Mao, Z., Yue, Y., & Rashmi, K. V. (2023). GL-Cache: Group-level learning for efficient and high-performance caching. 21st USENIX Conference on File and Storage Technologies. https://www.usenix.org/conference/fast23/presentation/yang-juncheng

Zhang, Y., Yang, J., Yue, Y., Vigfusson, Y., & Rashmi, K. V. (2024). SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches. 21st USENIX Symposium on Networked Systems Design and Implementation. https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo

Zhou, W., Niu, Z., Xiong, Y., Fang, J., & Wang, Q. (2025). 3L-Cache: Low Overhead and Precise Learning-based Eviction Policy for Caches. 23rd USENIX Conference on File and Storage Technologies. https://www.usenix.org/conference/fast25/presentation/zhou-wenbin

Zong, T., Li, C., Lei, Y., Li, G., Cao, H., & Liu, Y. (2021). Cocktail Edge Caching: Ride Dynamic Trends of Content Popularity with Ensemble Learning. arXiv (Cornell University). https://doi.org/10.48550/arxiv.2101.05885